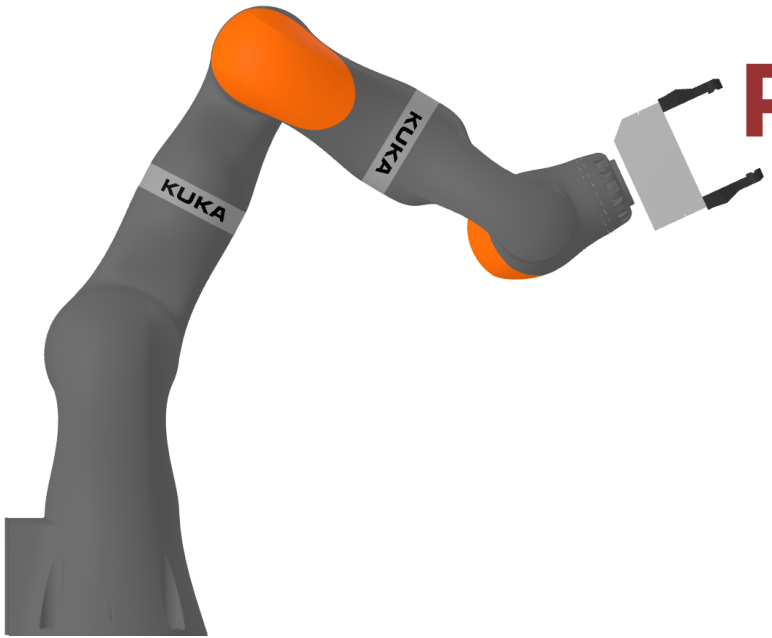


Reinforcement Learning

(Part 1)

MIT 6.4210/2:

Robotic Manipulation

Fall 2024, Lecture 18

Massachusetts Institute of Technology

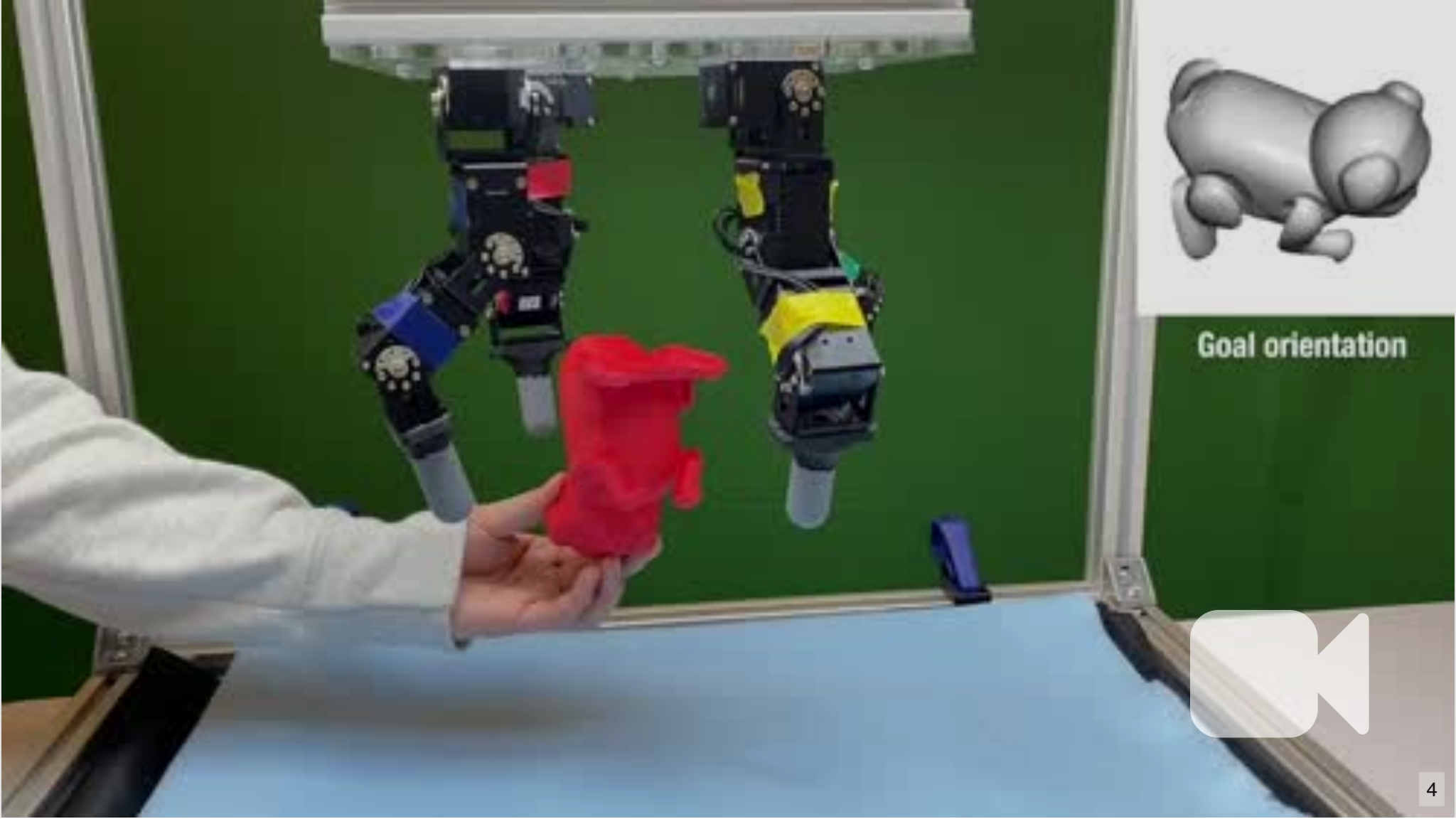


Recipe:

1. Make the simulator
2. Write cost function
3. Deep policy gradient

OpenAI - Learning Dexterity

https://www.youtube.com/embed/zXbb6KQ0xV8?mute=1&rel=0&enablejsapi=1


Goal orientation

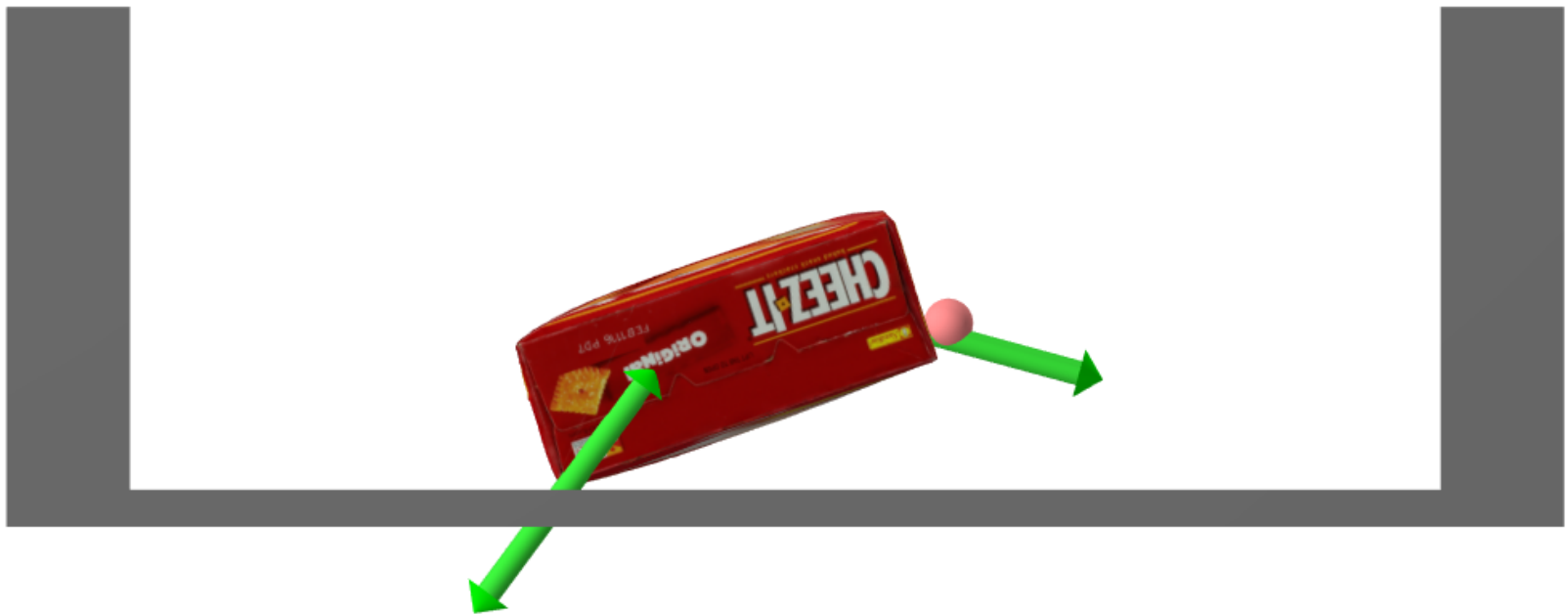
https://stable-baselines3.readthedocs.io/en/master/guide/algos.html



OpenAI - Learning Dexterity

"PPO has become the default reinforcement learning algorithm at OpenAI because of its ease of use and good performance."

https://openai.com/blog/openai-baselines-ppo/


CHEEZ-IT



Policy Architecture

Network

```
model = PPO('MlpPolicy', env, verbose=1, tensorboard_log=log)
```

```
stable_baselines3/common/policies.py#L435-L440

 # Default network architecture, from stable-baselines
net_arch = [dict(pi=[64, 64], vf=[64, 64])]
```

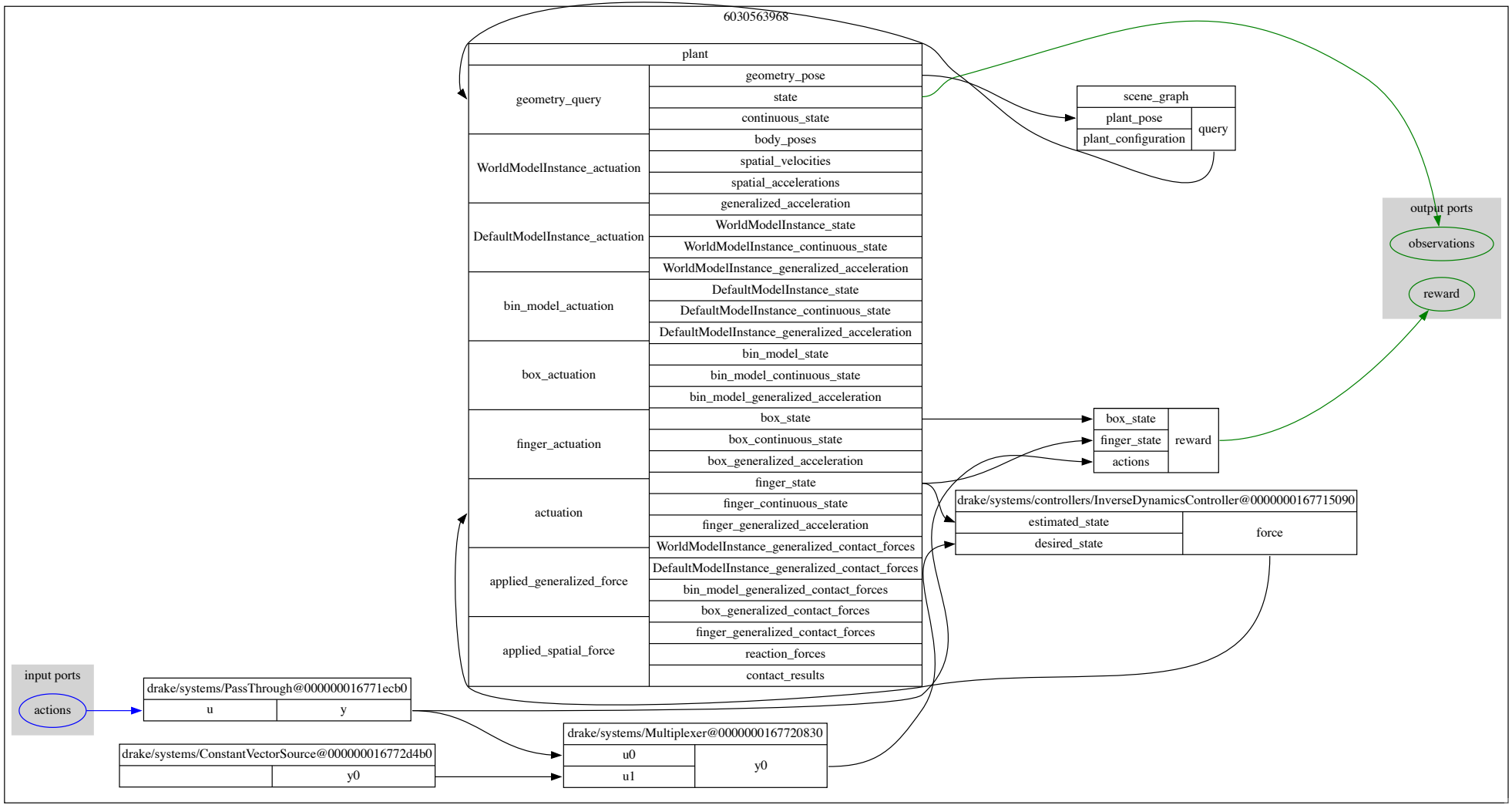
Actions

approximately:

```
builder.ExportOutput(inv_dynamics.get_desired_position(), "actions")
```

Observations

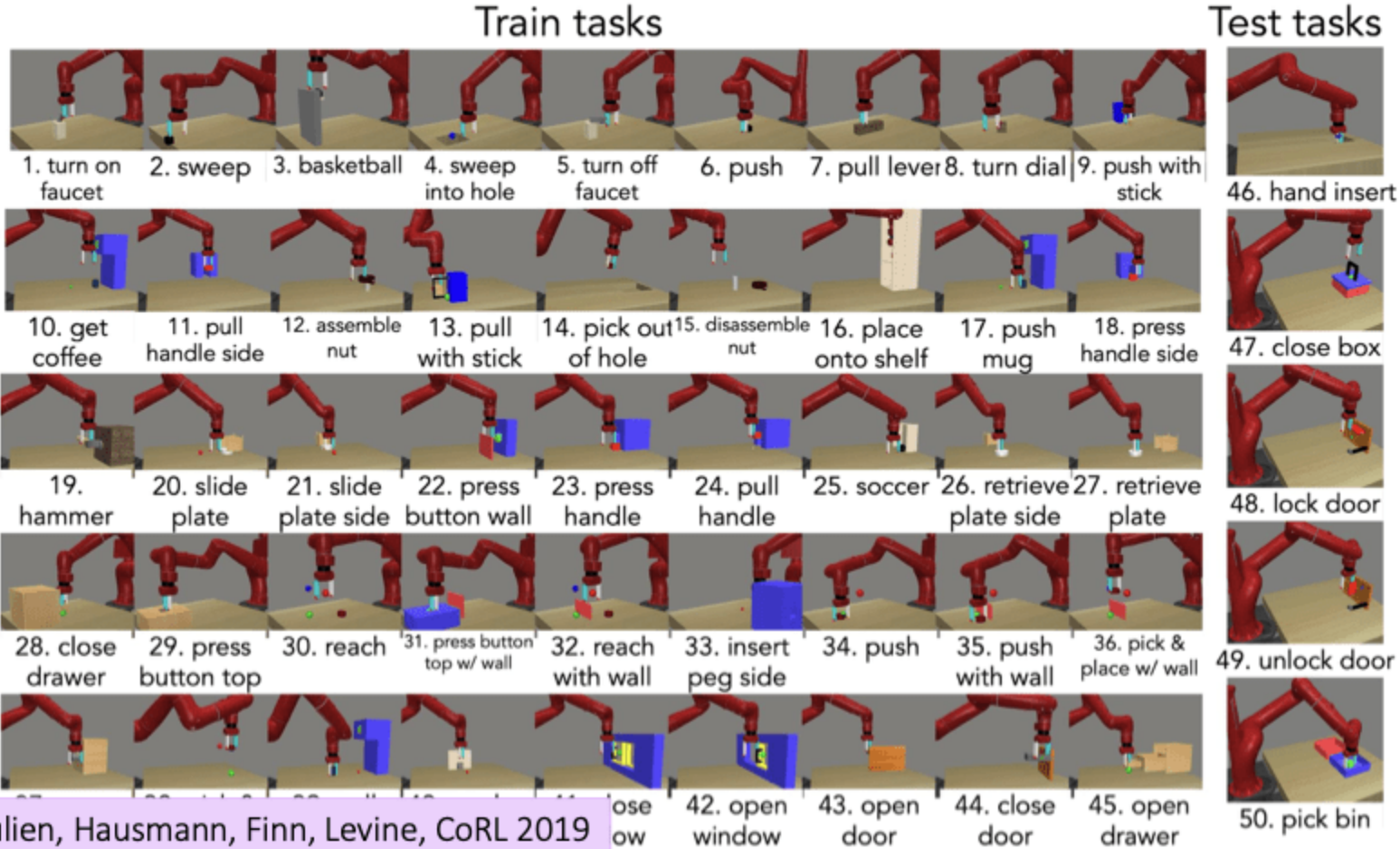
```
builder.ExportOutput(plant.get_state_output_port(), "observations")
```


6030563968
plant
geometry_query
geometry_pose
state
continuous_state
WorldModelInstance_actuation
body_poses
spatial_velocities
spatial_accelerations
generalized_acceleration
DefaultModelInstance_actuation
WorldModelInstance_state
WorldModelInstance_continuous_state
WorldModelInstance_generalized_acceleration
bin_model_actuation
DefaultModelInstance_state
DefaultModelInstance_continuous_state
DefaultModelInstance_generalized_acceleration
box_actuation
bin_model_state
bin_model_continuous_state
bin_model_generalized_acceleration
finger_actuation
box_state
box_continuous_state
box_generalized_acceleration
actuation
finger_state
finger_continuous_state
finger_generalized_acceleration
WorldModelInstance_generalized_contact_forces
applied_generalized_force
DefaultModelInstance_generalized_contact_forces
bin_model_generalized_contact_forces
box_generalized_contact_forces
applied_spatial_force
finger_generalized_contact_forces
reaction_forces
contact_results
scene_graph
plant_pose
plant_configuration
query
output ports
observations
reward
box_state
finger_state
actions
reward
drake/systems/controllers/InverseDynamicsController@0000000167715090
estimated_state
desired_state
force
input ports
actions
drake/systems/PassThrough@000000016771ecb0
u
y
drake/systems/ConstantVectorSource@000000016772d4b0
y0
drake/systems/Multiplexer@0000000167720830
u0
u1
y0

Cost Function

```
angle_from_vertical = (box_state[2] % np.pi) - np.pi / 2
cost = 2 * angle_from_vertical**2  # box angle
cost += 0.1 * box_state[5]**2  # box velocity
effort = actions - finger_state[:2]
cost += 0.1 * effort.dot(effort)  # effort
# finger velocity
cost += 0.1 * finger_state[2:].dot(finger_state[2:])
# Add 10 to make rewards positive (to avoid rewarding simulator
# crashes).
output[0] = 10 - cost
```

MetaWorld: suite of RL problems designed for meta-learning



Yu, Quillen, He, Julien, Hausmann, Finn, Levine, CoRL 2019

Reward

```
def compute_reward(self, actions, obs, mode = 'general'):
    if isinstance(obs, dict):
        obs = obs['state_observation']

    objPos = obs[3:6]

    rightFinger, leftFinger = self.get_site_pos('rightEndEffector'),
                              self.get_site_pos('leftEndEffector')
    fingerCOM  =  (rightFinger + leftFinger)/2

    heightTarget = self.heightTarget
    placingGoal = self._state_goal

    reachDist = np.linalg.norm(objPos - fingerCOM)
    placingDist = np.linalg.norm(objPos[:2] - placingGoal[:-1])

    def reachReward():
        reachRew = -reachDist# + min(actions[-1], -1)/50
        reachDistxy = np.linalg.norm(objPos[:-1] - fingerCOM[:-1])
        zRew = np.linalg.norm(fingerCOM[-1] - self.init_fingerCOM[-1])
        if reachDistxy < 0.06: #0.02
            reachRew = -reachDist
        else:
            reachRew =  -reachDistxy - zRew
        #incentive to close fingers when reachDist is small
        if reachDist < 0.05:
            reachRew = -reachDist + max(actions[-1],0)/50
        return reachRew , reachDist

    def pickCompletionCriteria():
        tolerance = 0.01
        if objPos[2] >= (heightTarget- tolerance):
            return True
        else:
            return False

    if pickCompletionCriteria():
        self.pickCompleted = True

    def objDropped():
            return (objPos[2] < (self.objHeight + 0.005)) and
(placingDist >0.02) and (reachDist > 0.02)
            # Object on the ground, far away from the goal,
and from the gripper
            #Can tweak the margin limits

        def objGrasped(thresh = 0):
            sensorData = self.data.sensordata
            return (sensorData[0]>thresh) and
(sensorData[1]> thresh)

        def placeCompletionCriteria():
            if abs(objPos[0] - placingGoal[0]) < 0.05 and \
                abs(objPos[1] - placingGoal[1]) < 0.05 and \
                objPos[2] < self.objHeight + 0.05:
                return True
            else:
                return False
        if placeCompletionCriteria():
            self.placeCompleted = True

        def orig_pickReward():
            # hScale = 50
            hScale = 100
            if self.placeCompleted or (self.pickCompleted
and not(objDropped())):
                return hScale*heightTarget
            # elif (reachDist < 0.1) and (objPos[2]>
(self.objHeight + 0.005)) :
            elif (reachDist < 0.1) and (objPos[2]>
(self.objHeight + 0.005)) :
                return hScale* min(heightTarget, objPos[2])
            else:
                return 0

        Dist]
```

```
def general_pickReward():
        hScale = 50
        if self.placeCompleted or (self.pickCompleted and objGrasped()):
            return hScale*heightTarget
        elif objGrasped() and (objPos[2]> (self.objHeight + 0.005)):
            return hScale* min(heightTarget, objPos[2])
        else:
            return 0

    def placeReward():
        # c1 = 1000 ; c2 = 0.01 ; c3 = 0.001
        c1 = 1000 ; c2 = 0.01 ; c3 = 0.001
        placeRew = 1000*(self.maxPlacingDist - placingDist) + c1*(np.exp(-
(placingDist**2)/c2) + np.exp(-(placingDist**2)/c3))
        placeRew = max(placeRew,0)
        if mode == 'general':
            cond = self.pickCompleted and objGrasped()
        else:
            cond = self.pickCompleted and (reachDist < 0.1) and
not(objDropped())
        if self.placeCompleted:
            return [-200*actions[-1] + placeRew, placingDist]
        elif cond:
            if abs(objPos[0] - placingGoal[0]) < 0.05 and \
                abs(objPos[1] - placingGoal[1]) < 0.05:
                return [-200*actions[-1] + placeRew, placingDist]
            else:
                return [placeRew, placingDist]
        else:
            return [0 , placingDist

    reachRew, reachDist = reachReward()
    if mode == 'general':
        pickRew = general_pickReward()
    else:
        pickRew = orig_pickReward()
    placeRew , placingDist = placeReward()
    # assert ((placeRew >=0) and (pickRew>=0))
    if self.placeCompleted:
        reachRew = 0
        reachDist = 0
    reward = reachRew + pickRew + placeRew
    return [reward, reachRew, reachDist, pickRew, placeRew, placing
```

Reward for pick and place task

hand close to obj	obj grasped	pick ever completed	hand close to goal	place ever completed	reward
0	0	*	*	0	-distXY(hand, obj) – distZ(hand,obj)
1	0	0	*	0	-dist(hand, obj) + c1 * closeCmd
1	1	0	*	0	-dist(hand,obj) + c1 * closeCmd + c2 * min(obj.z, targetHt)
1	1	1	*	0	-dist(hand,obj) + c1 * closeCmd + c2 * targetHt + placeRew
1	1	1	0	0	-dist(hand,obj) + c1 * closeCmd + c2 * targetHt + placeRew + c3 * openCmd
*	*	*	*	1	c2 * targetHt + placeRew + c3 * openCmd

place ever completed: |objx – goalx| < c4 and |objy-goaly|< c5 and |objz – goalz| < c6
pick ever completed: objz > targetHt + c7
obj grasp: test on current finger sensors
hand close to obj, hand close to goal: constant x,y,z thresholds
placeDist = |objxy – goalxy|
placeRew = max(0, c8*(c9- placingDist) + c10*(exp(-(placingDist**2)/c11) +exp(-(placingDist**2)/c12)))

Reward for pick and place task

hand close to obj	obj grasped	pick ever completed	hand close to goal	place ever completed	reward
0	0	*	*	0	move toward object
1	0	0	*	0	move toward object + close fingers
1	1	0	*	0	move toward object + close fingers + raise
1	1	1	*	0	move toward object + close fingers + move toward goal
1	1	1	0	0	move toward object + close fingers + move toward goal + open fingers
*	*	*	*	1	move toward goal + open fingers

place ever completed: |objx – goalx| < c4 and |objy-goaly|< c5 and |objz – goalz| < c6
pick ever completed: objz > targetHt + c7
obj grasp: test on current finger sensors
hand close to obj, hand close to goal: constant x,y,z thresholds
placeDist = |objxy – goalxy|
placeRew = max(0, c8*(c9- placingDist) + c10*(exp(-(placingDist**2)/c11) +exp(-(placingDist**2)/c12)))

Intro to RL & MDP (Abbeel)

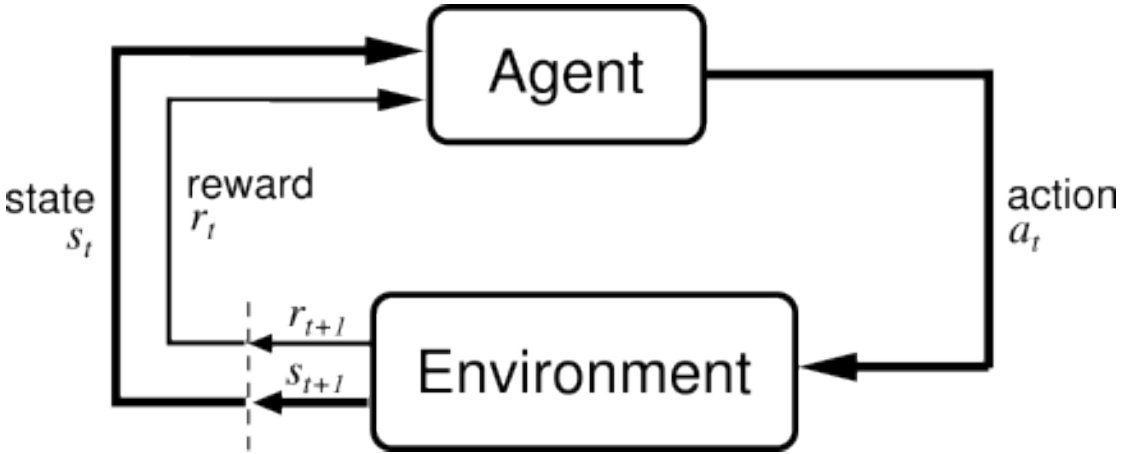
Deep Reinforcement Learning
Lecture 1: Motivation + Overview + MDPs + Exact Solution Methods

Pieter Abbeel

OpenAI / UC Berkeley / Gradescope

Many slides made with John Schulman, Yan (Rocky) Duan and Xi (Peter) Chen

Markov Decision Process



Assumption: agent gets to observe the state

[Drawing from Sutton and Barto, Reinforcement Learning: An Introduction, 1998]

Some Reinforcement Learning Success Stories



Kohl and Stone, 2004



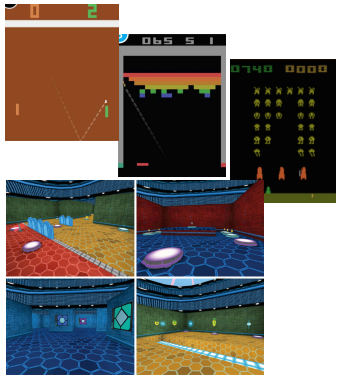
Ng et al, 2004



Tedrake et al, 2005



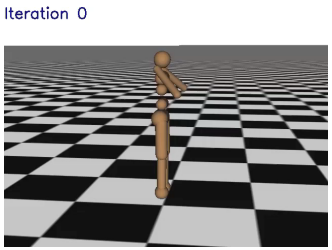
Kober and Peters, 2009



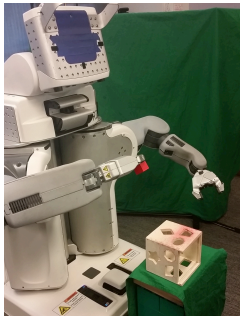
Mnih et al 2013 (DQN)
Mnih et al, 2015 (A3C)



Silver et al, 2014 (DPG)
Lillicrap et al, 2015 (DDPG)



Schulman et al,
2016 (TRPO + GAE)

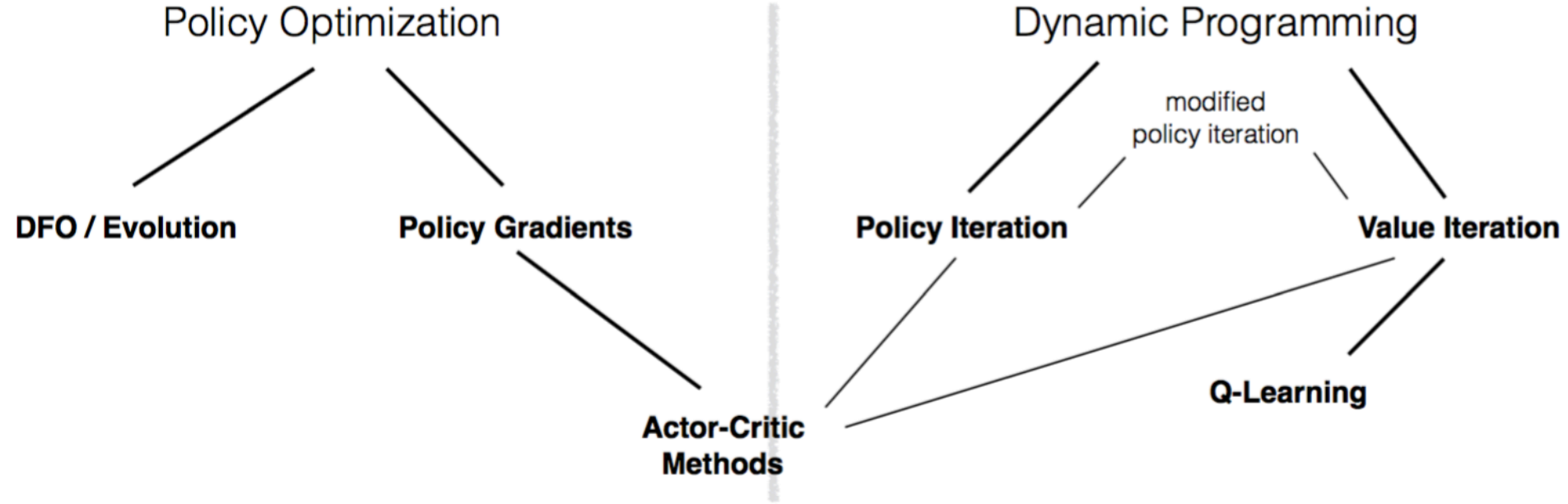


Levine*, Finn*, et
al, 2016
(GPS)

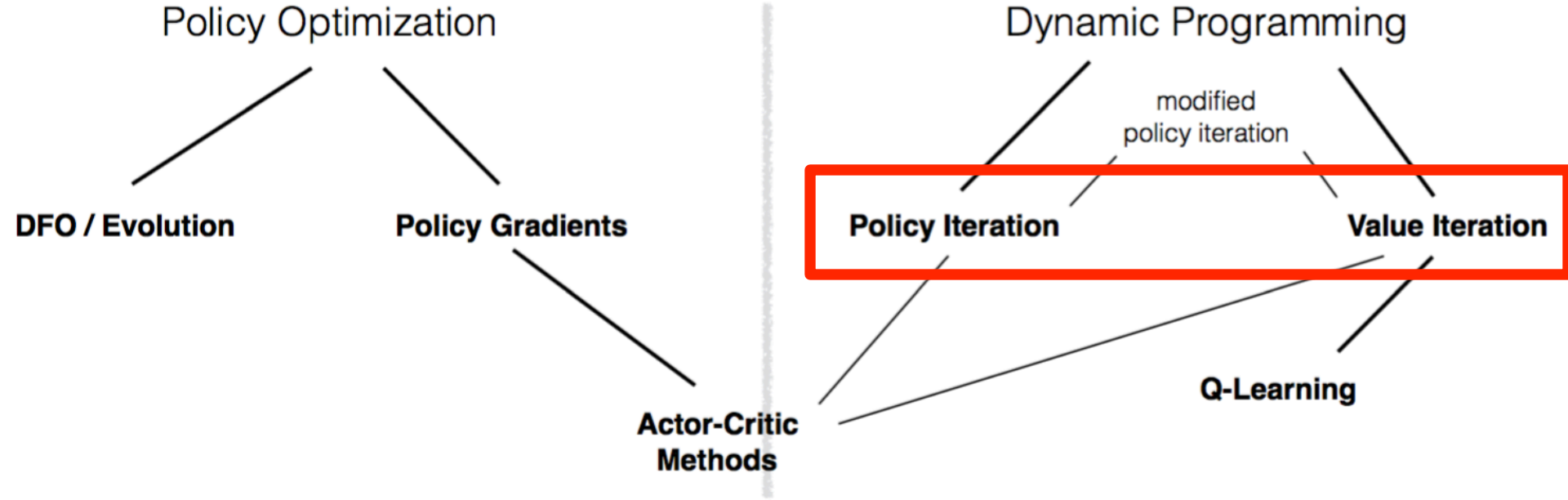


Silver*, Huang*, et
al, 2016
(AlphaGo)

RL Algorithms Landscape



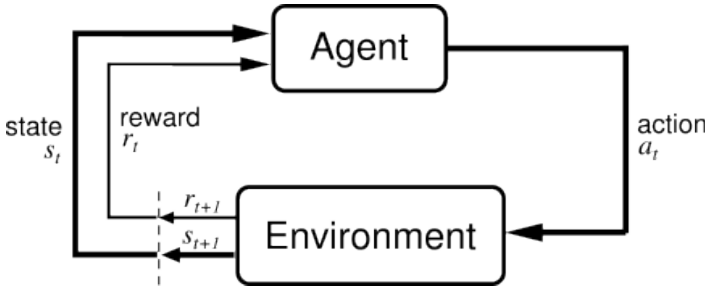
RL Algorithms Landscape



Markov Decision Process (MDP)

An MDP is defined by:

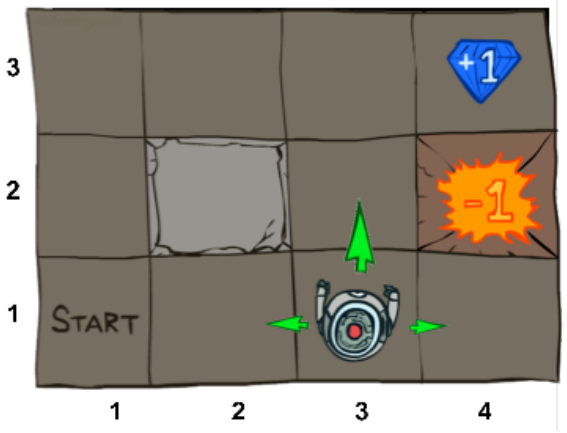
- Set of states S
- Set of actions A
- Transition function $P(s' \mid s, a)$
- Reward function $R(s, a, s')$
- Start state s_0
- Discount factor γ
- Horizon H



Example MDP: Gridworld

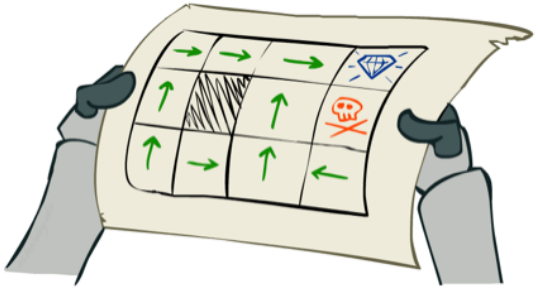
An MDP is defined by:

- Set of states S
- Set of actions A
- Transition function $P(s' \mid s, a)$
- Reward function $R(s, a, s')$
- Start state s_0
- Discount factor γ
- Horizon H



Goal: $max_\pi \mathrm{E}[\sum_{t=0}^{H} \gamma^t R(S_t, A_t, S_{t+1}) | \pi]$

π:



Outline

- Optimal Control

  =

  given an MDP (S, A, P, R, γ, H)

  find the optimal policy π^*

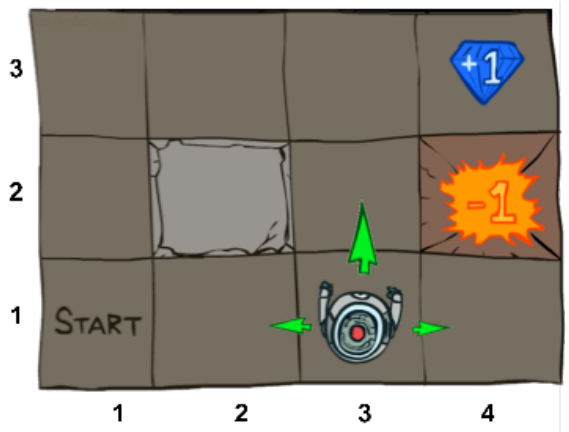
- Exact Methods:
  - ***Value Iteration***
  - Policy Iteration

For now: small discrete state-action spaces as they are simpler to get the main concepts across. We will consider large / continuous spaces later!

Optimal Value Function V*

$$V^*(s) = \max_{\pi} \mathbb{E}\left[\sum_{t=0}^{H} \gamma^t R(s_t, a_t, s_{t+1}) \mid \pi, s_0 = s\right]$$

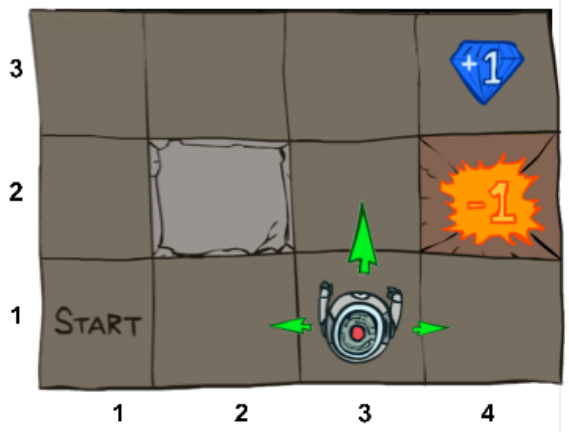
= sum of discounted rewards when starting from state s and acting optimally



Optimal Value Function V*

$$V^*(s) = \max_{\pi} \mathbb{E}\left[\sum_{t=0}^{H} \gamma^t R(s_t, a_t, s_{t+1}) \mid \pi, s_0 = s\right]$$

= sum of discounted rewards when starting from state s and acting optimally



Let's assume:

actions deterministically successful, gamma = 1, H = 100

V*(4,3) =

V*(3,3) =

V*(2,3) =

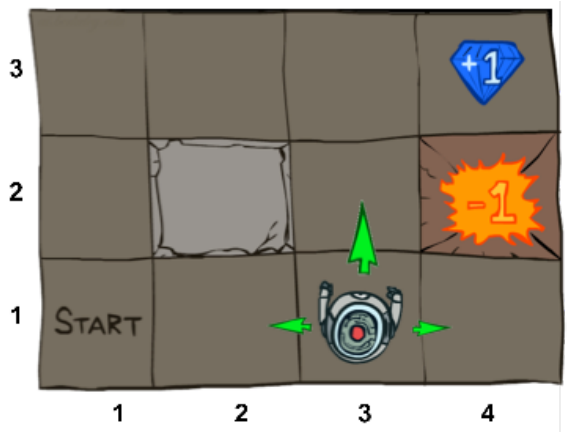
V*(1,1) =

V*(4,2) =

Optimal Value Function V*

$$V^*(s) = \max_{\pi} \mathbb{E}\left[\sum_{t=0}^{H} \gamma^t R(s_t, a_t, s_{t+1}) \mid \pi, s_0 = s\right]$$

= sum of discounted rewards when starting from state s and acting optimally



Let's assume:

actions deterministically successful, gamma = 0.9, H = 100

V*(4,3) =

V*(3,3) =

V*(2,3) =

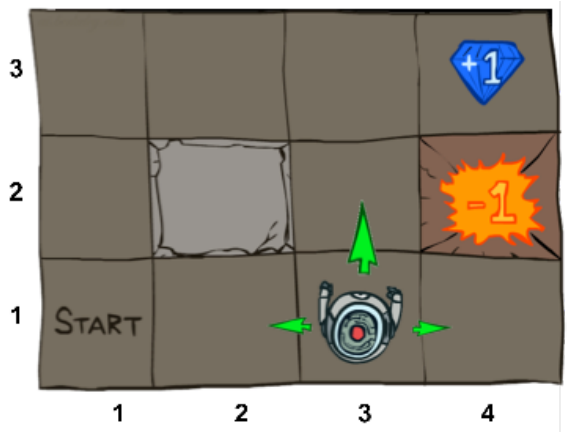
V*(1,1) =

V*(4,2) =

Optimal Value Function V*

$$V^*(s) = \max_{\pi} \mathbb{E}\left[\sum_{t=0}^{H} \gamma^t R(s_t, a_t, s_{t+1}) \mid \pi, s_0 = s\right]$$

= sum of discounted rewards when starting from state s and acting optimally



Let's assume:

actions successful w/probability 0.8, gamma = 0.9, H = 100

V*(4,3) =

V*(3,3) =

V*(2,3) =

V*(1,1) =

V*(4,2) =

Value Iteration

- $V_0^*(s)$ = optimal value for state s when H=0
  - $V_0^*(s) = 0 \quad \forall s$
- $V_1^*(s)$ = optimal value for state s when H=1
  - $V_1^*(s) = \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_0^*(s'))$
- $V_2^*(s)$ = optimal value for state s when H=2
  - $V_2^*(s) = \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_1^*(s'))$
- $V_k^*(s)$ = optimal value for state s when H = k
  - $V_k^*(s) = \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_{k-1}^*(s'))$

Value Iteration

Algorithm:

Start with $V_0^*(s) = 0$ for all s.

For k = 1, … , H:

For all states s in S:

$$V_k^*(s) \leftarrow \max_a \sum_{s'} P(s'|s,a) \left(R(s,a,s') + \gamma V_{k-1}^*(s')\right)$$

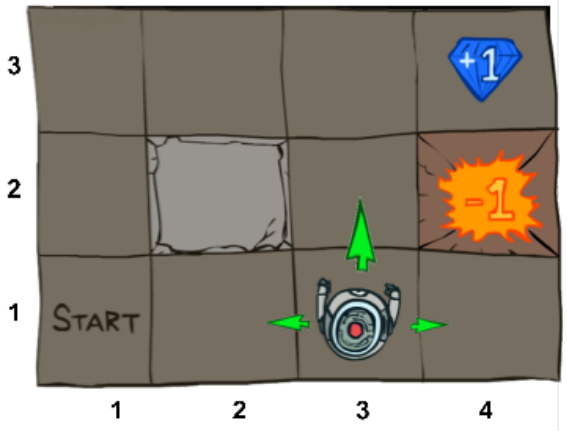
$$\pi_k^*(s) \leftarrow \arg\max_a \sum_{s'} P(s'|s,a) \left(R(s,a,s') + \gamma V_{k-1}^*(s')\right)$$

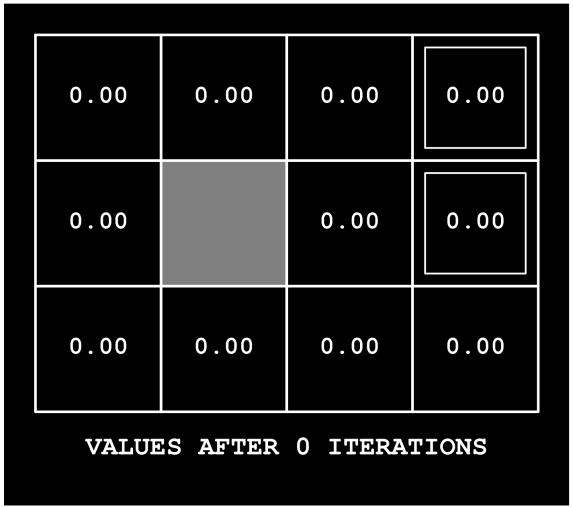
This is called a value update or Bellman update/back-up

Value Iteration

$$V_0(s) \leftarrow 0$$

k = 0

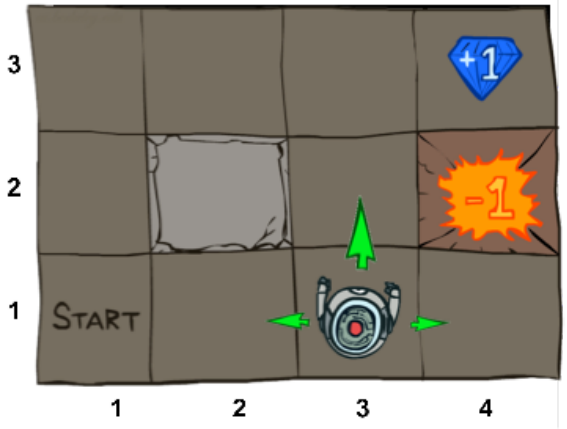




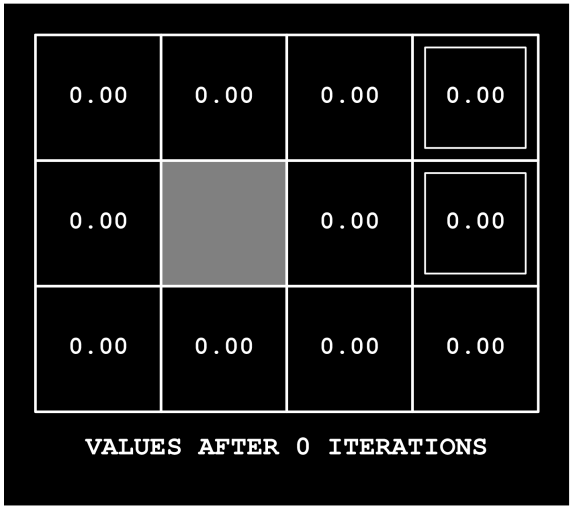
Noise = 0.2
Discount = 0.9

Value Iteration

$$V_1(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_0(s'))$$



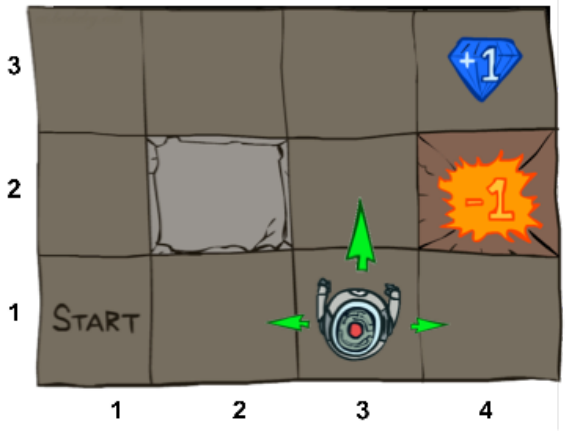
k = 0



Noise = 0.2
Discount = 0.9

Value Iteration

$$V_2(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_1(s'))$$



k = 1



Noise = 0.2
Discount = 0.9

Value Iteration

$$V_2(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_1(s'))$$

k = 2



Noise = 0.2
Discount = 0.9

Value Iteration

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

k = 3



Noise = 0.2
Discount = 0.9

Value Iteration

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

k = 4



Noise = 0.2
Discount = 0.9

Value Iteration

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

k = 5



Noise = 0.2
Discount = 0.9

Value Iteration

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

k = 6



Noise = 0.2
Discount = 0.9

Value Iteration

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

k = 7



Noise = 0.2
Discount = 0.9

Value Iteration

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

k = 8



Noise = 0.2
Discount = 0.9

Value Iteration

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

k = 9



Noise = 0.2
Discount = 0.9

Value Iteration

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

k = 10



Noise = 0.2
Discount = 0.9

Value Iteration

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

k = 11



Noise = 0.2
Discount = 0.9

Value Iteration

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

k = 12



Noise = 0.2
Discount = 0.9

Value Iteration

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

k = 100



Noise = 0.2
Discount = 0.9

Value Iteration Convergence

Theorem. *Value iteration converges. At convergence, we have found the optimal value function V* for the discounted infinite horizon problem, which satisfies the Bellman equations*

$$\forall S \in S: \quad V^*(s) = \max_a \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma V^*(s')\right]$$

- Now we know how to act for infinite horizon with discounted rewards!
  - Run value iteration till convergence.
  - This produces V*, which in turn tells us how to act, namely following:

$$\pi^*(s) = \arg\max_a \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma V^*(s')\right]$$

- Note: the infinite horizon optimal policy is stationary, i.e., the optimal action at a state s is the same action at all times. (Efficient to store!)

Convergence: Intuition

- $V^*(s)$ = expected sum of rewards accumulated starting from state s, acting optimally for ∞ steps
- $V_H^*(s)$ = expected sum of rewards accumulated starting from state s, acting optimally for H steps

- Additional reward collected over time steps H+1, H+2, …

$$\gamma^{H+1}R(s_{H+1}) + \gamma^{H+2}R(s_{H+2}) + \ldots \leq \gamma^{H+1}R_{max} + \gamma^{H+2}R_{max} + \ldots = \frac{\gamma^{H+1}}{1-\gamma}R_{max}$$

goes to zero as H goes to infinity

Hence $V_H^* \xrightarrow{H\to\infty} V*$

For simplicity of notation in the above it was assumed that rewards are always greater than or equal to zero. If rewards can be negative, a similar argument holds, using max |R| and bounding from both sides.

Convergence and Contractions

- Define the max-norm: $||U|| = \max_s |U(s)|$

- Theorem: For any two approximations U and V

$$||U_{i+1} - V_{i+1}|| \leq \gamma \, ||U_i - V_i||$$

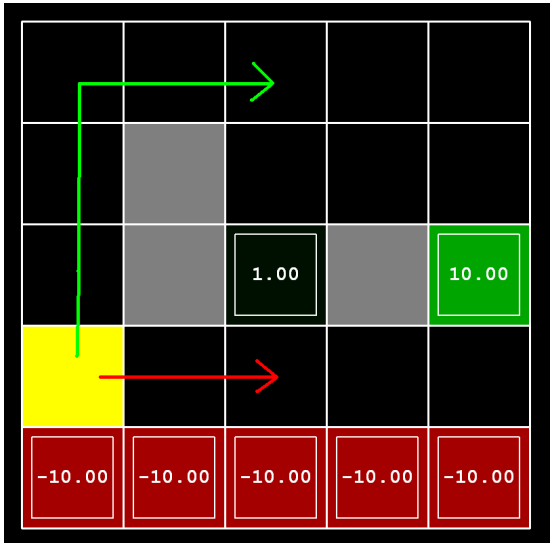
  - I.e., any distinct approximations must get closer to each other, so, in particular, any approximation must get closer to the true U and value iteration converges to a unique, stable, optimal solution

- Theorem:

$$||V_{i+1} - V_i|| < \epsilon, \Rightarrow ||V_{i+1} - V^*|| < 2\epsilon\gamma/(1-\gamma)$$

  - I.e. once the change in our approximation is small, it must also be close to correct

Exercise 1: Effect of Discount and Noise



(a) Prefer the close exit (+1), risking the cliff (-10)

(b) Prefer the close exit (+1), but avoiding the cliff (-10)

(c) Prefer the distant exit (+10), risking the cliff (-10)

(d) Prefer the distant exit (+10), avoiding the cliff (-10)

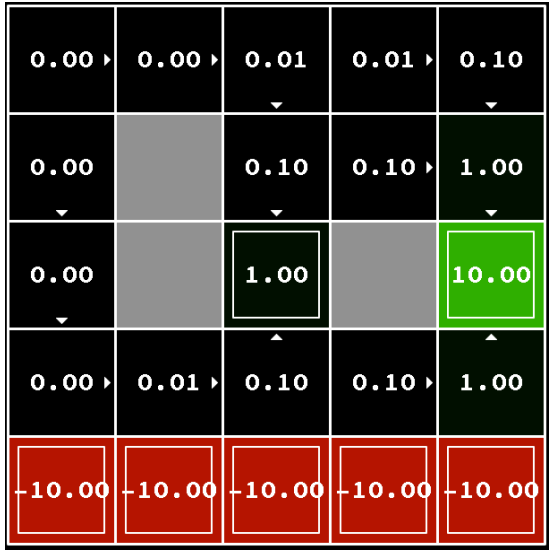
(1) $\gamma = 0.1$, noise = 0.5

(2) $\gamma = 0.99$, noise = 0

(3) $\gamma = 0.99$, noise = 0.5

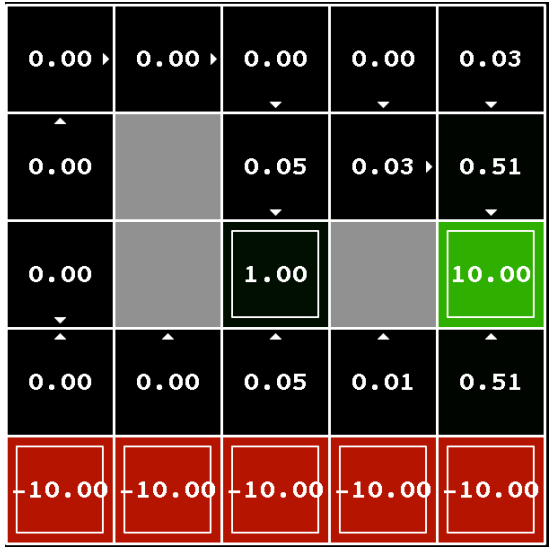
(4) $\gamma = 0.1$, noise = 0

Exercise 1 Solution



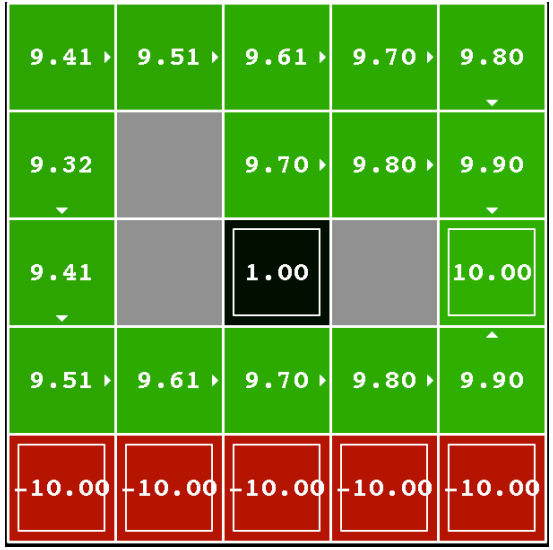
(a) Prefer close exit (+1), risking the cliff (-10) --- (4) γ = 0.1, noise = 0

Exercise 1 Solution



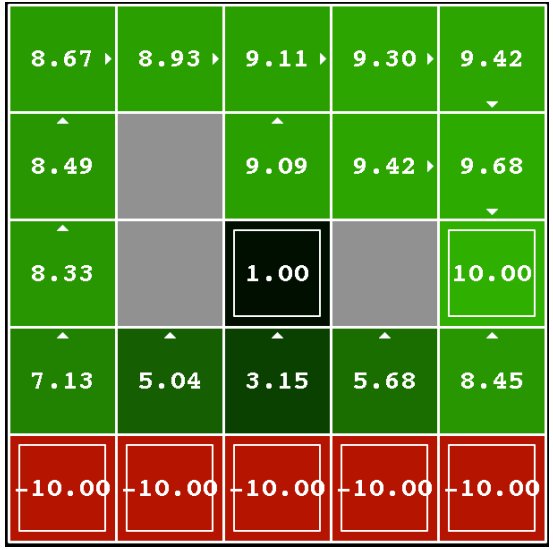
(b) Prefer close exit (+1), avoiding the cliff (-10) --- (1) $\gamma = 0.1$, noise = 0.5

Exercise 1 Solution



(c) Prefer distant exit (+1), risking the cliff (-10) --- (2) γ = 0.99, noise = 0

Exercise 1 Solution



(d) Prefer distant exit (+1), avoid the cliff (-10) --- (3) γ = 0.99, noise = 0.5

Q-Values

$Q^*(s, a)$ = expected utility starting in s, taking action a, and (thereafter) acting optimally

Bellman Equation:

$$Q^*(s,a) = \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma \max_{a'} Q^*(s',a'))$$

Q-Value Iteration:

$$Q^*_{k+1}(s,a) \leftarrow \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma \max_{a'} Q^*_k(s',a'))$$

Q-Value Iteration

$$Q^*_{k+1}(s,a) \leftarrow \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma \max_{a'} Q^*_k(s',a'))$$

k = 100



Noise = 0.2

Discount = 0.9

Outline

- Optimal Control

  =

  given an MDP (S, A, P, R, γ, H)

  find the optimal policy π^*

- Exact Methods:
  - ***Value Iteration***
  - Policy Iteration

For now: small discrete state-action spaces as they are simpler to get the main concepts across. We will consider large / continuous spaces later!

Recap Q-Values

$Q^*(s, a)$ = expected utility starting in s, taking action a, and (thereafter) acting optimally

Bellman Equation:

$$Q^*(s,a) = \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma \max_{a'} Q^*(s',a'))$$

Q-Value Iteration:

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma \max_{a'} Q_k(s',a'))$$

(Tabular) Q-Learning

- Q-value iteration: $Q_{k+1}(s,a) \leftarrow \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma \max_{a'} Q_k(s',a'))$
- Rewrite as expectation: $Q_{k+1} \leftarrow \mathbb{E}_{s' \sim P(s'|s,a)} \left[R(s,a,s') + \gamma \max_{a'} Q_k(s',a') \right]$
- (Tabular) Q-Learning: replace expectation by samples
  - For an state-action pair (s,a), receive: $s' \sim P(s'|s,a)$
  - Consider your old estimate: $Q_k(s,a)$
  - Consider your new sample estimate:

$$\text{target}(s') = R(s,a,s') + \gamma \max_{a'} Q_k(s',a')$$

  - Incorporate the new estimate into a running average:

$$Q_{k+1}(s,a) \leftarrow (1-\alpha) Q_k(s,a) + \alpha \left[\text{target}(s') \right]$$

(Tabular) Q-Learning

Algorithm:

Start with $Q_0(s, a)$ for all s, a.

Get initial state s

For k = 1, 2, … till convergence

Sample action a, get next state s'

If s' is terminal:

$$\text{target} = R(s, a, s')$$

Sample new initial state s'

else:

$$\text{target} = R(s, a, s') + \gamma \max_{a'} Q_k(s', a')$$

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha) Q_k(s, a) + \alpha \, [\text{target}]$$

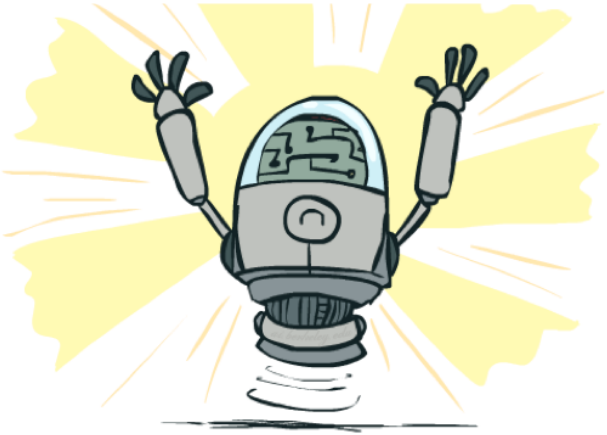
$$s \leftarrow s'$$

How to sample actions?

- Choose random actions?
- Choose action that maximizes $Q_k(s, a)$ (i.e. greedily)?
- ε-Greedy: choose random action with prob. ε, otherwise choose action greedily

Q-Learning Properties

- Amazing result: Q-learning converges to optimal policy -- even if you're acting suboptimally!
- This is called off-policy learning
- Caveats:
  - You have to explore enough
  - You have to eventually make the learning rate small enough
  - … but not decrease it too quickly



Q-Learning Properties

- Technical requirements.
  - All states and actions are visited infinitely often
    - Basically, in the limit, it doesn't matter how you select actions (!)
  - Learning rate schedule such that for all state and action pairs (s,a):

$$\sum_{t=0}^{\infty} \alpha_t(s,a) = \infty \qquad \sum_{t=0}^{\infty} \alpha_t^2(s,a) < \infty$$

For details, see Tommi Jaakkola, Michael I. Jordan, and Satinder P. Singh. On the convergence of stochastic iterative dynamic programming algorithms. Neural Computation, 6(6), November 1994.

Policy Iteration Guarantees

Policy Iteration iterates over:

- Policy evaluation for current policy π_k :
  - Iterate until convergence

$$V_{i+1}^{\pi_k}(s) \leftarrow \sum_{s'} P(s'|s, \pi_k(s)) \left[R(s, \pi(s), s') + \gamma V_i^{\pi_k}(s')\right]$$

- Policy improvement: find the best action according to one-step look-ahead

$$\pi_{k+1}(s) \leftarrow \arg\max_a \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma V^{\pi_k}(s')\right]$$

Theorem. Policy iteration is guaranteed to converge and at convergence, the current policy and its value function are the optimal policy and the optimal value function!

Proof sketch:

(1) Guarantee to converge: In every step the policy improves. This means that a given policy can be encountered at most once. This means that after we have iterated as many times as there are different policies, i.e., (number actions)$^{(\text{number states})}$, we must be done and hence have converged.

(2) Optimal at convergence: by definition of convergence, at convergence $\pi_{k+1}(s) = \pi_k(s)$ for all states s. This means $\forall s \;\; V^{\pi_k}(s) = \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_i^{\pi_k}(s')\right]$

Hence V^{π_k} satisfies the Bellman equation, which means V^{π_k} is equal to the optimal value function V*.